



Cryptography mini-project #11

Theme:

A Secure 64-bit SPN-Based Block Cipher: Design and Analysis

Realised by :

❖ Hassi Imad-Eddine

Supervisor :

❖ Dr. Kaddach

Contents

2.1. Description of the Core Design Idea-----	2
3.1. Order of Operations in Encryption-----	4
3.2. Mathematical Representation -----	4
4.1. S-Box: Structure and Rationale-----	5
4.2. P-Box: Structure and Rationale-----	6
5.1. Order of Operations -----	6
5.2. Pseudocode: Single Round Transformation -----	7
5.3. Key Derivation Rationale-----	7
5.4. Pseudocode: Key Schedule-----	8
6.1. Implementation Screenshots -----	10
7.1. Test Vector Setup -----	13
7.2. Plaintext and Ciphertext Results -----	14
8.1. The Development Process -----	15
8.2. Challenges Encountered-----	15
8.3. Observations -----	15
FIGURE 1 : SYMMETRIC SPN STRUCTURE SHOWING ENCRYPTION (LEFT) AND DECRYPTION (RIGHT) PATHS	3
FIGURE 2 : STRUCTURE OF A SINGLE ENCRYPTION ROUND.....	4
FIGURE 3 : S-BOX CONSTRUCTION USING $GF(2^8)$ MULTIPLICATIVE INVERSE	5
FIGURE 4 : P-BOX STRUCTURE USING 8x8 MATRIX TRANSPOSE	6
FIGURE 5 : KEY SCHEDULE GENERATING 9 SUBKEYS USING ROL64 + CONSTANT ADDITION	8
FIGURE 6 : $GF(2^8)$ ARITHMETIC AND S-BOX GENERATION CODE	10
FIGURE 7 : P-BOX GENERATION, LAYER APPLICATION, AND MIX_XOR IMPLEMENTATION	11
FIGURE 8 : CORE ENCRYPTION, DECRYPTION, AND KEY SCHEDULE FUNCTIONS.....	11
FIGURE 9 : TEST RESULTS SHOWING ALL 20/20 TEST VECTORS PASSED	12
FIGURE 10 : TEST VECTOR SETUP WITH MASTER KEY INITIALIZATION	13
TABLE 1 : ALGORITHM EVALUATION SUMMARY (PROJECT #11).....	9

1. Introduction

Project #11 involves the design and implementation of a 64-bit Substitution-Permutation Network (SPN) cipher written in Python. The main goal is to build a working symmetric encryption system that keeps data confidential by applying multiple rounds of bit-level transformations.

The cipher follows a modular bottom-up design philosophy. It operates on 64-bit data blocks using a 64-bit master key. All cryptographic components, including the S-Box and P-Box, are generated through direct mathematical computation rather than being hardcoded as static tables. This approach ensures the implementation is transparent and mathematically well-grounded.

The project specifications require a block size of 64 bits, a key size of 64 bits, an SPN structure with 8 rounds, a student-defined S-Box and P-Box, a MIX_XOR function as the mixing layer, and the ADD_KEY operation for subkey insertion. This report documents the design decisions, algorithm structure, implementation details, and test results.

2. Algorithm Concept

2.1. Description of the Core Design Idea

The cipher is built around an 8-round SPN architecture. Each round is designed to balance confusion and diffusion. Confusion makes the relationship between the key and the ciphertext as complex as possible, while diffusion spreads the influence of individual plaintext bits across the entire 64-bit block.

The design is structured around four core components:

- **Key Scheduling (ADD-ROT):** Rather than using the master key directly in every round, the key is transformed before each round using a 64-bit left rotation by 9 positions combined with a modular addition of a round constant derived from the golden ratio ($0x9E3779B97F4A7C15 + i$). This ensures each of the 9 subkeys is different and avoids any linear relationship between consecutive round keys.
- **Mathematical Substitution (S-Box):** The 8-bit S-Box provides the non-linear confusion layer. It is constructed by computing the multiplicative inverse of each element in $GF(2^8)$ using the irreducible polynomial $x^8 + x^4 + x^3 + x + 1$ (0x11B). Each of the 8 bytes in the 64-bit state goes through this substitution independently. This provides strong nonlinearity and makes the cipher resistant to linear and differential cryptanalysis.
- **Linear Bit Permutation (P-Box):** The 64-bit P-Box achieves diffusion by reordering bits using a matrix transpose method. Bit at position $row \times 8 + col$ is sent to position $col \times 8 + row$, effectively transposing an 8×8 matrix of bits. This guarantees that bits from a single byte are spread to eight different byte positions in the next round, which maximizes the inter-byte avalanche effect.
- **Mixing Layer (MIX_XOR):** The MIX_XOR function provides additional diffusion within each round. It splits the 64-bit state into two 32-bit halves (L and R), then applies cross-mixing operations using 32-bit rotations. Specifically: $L \oplus= R$, then $R \oplus= ROL32(L, 11)$, then $L \oplus= ROL32(R, 23)$. This is applied in all 8 rounds, and the final key addition follows after the 8th round.

Together, these four components form a cipher that achieves a high degree of complexity with efficient computation. The structure is also symmetric, meaning decryption is simply the inverse of encryption applied in reverse order.

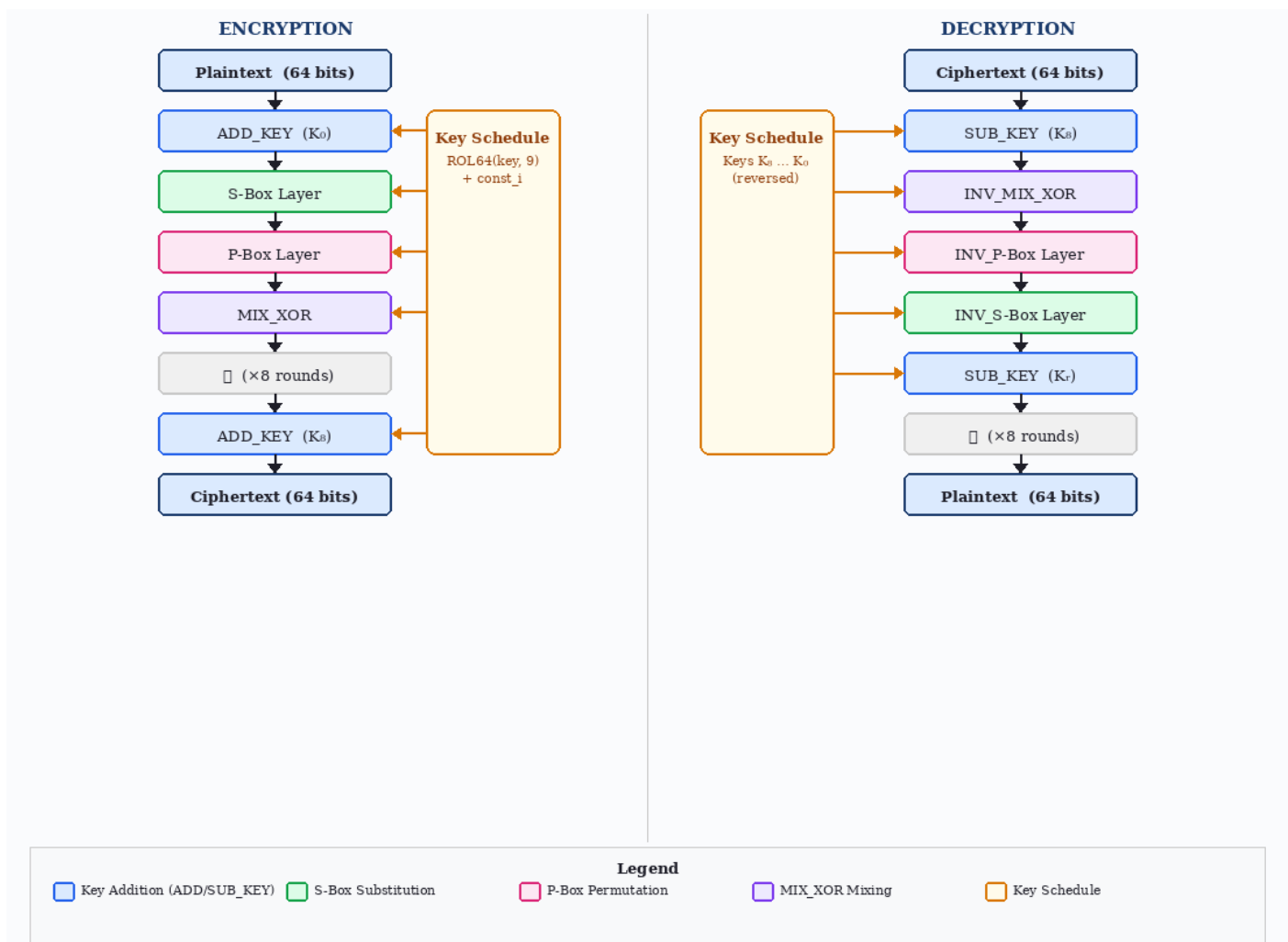


Figure 1 : Symmetric SPN structure showing Encryption (left) and Decryption (right) paths

As shown in Figure 1, the cipher uses a fully symmetric SPN structure. The encryption path follows 8 identical rounds, each consisting of ADD_KEY, S-Box, P-Box, and MIX_XOR, followed by a final key addition. The decryption path reverses the order and uses the inverse of each operation.

3. Round Structure

The cipher consists of 8 iterative rounds followed by one final subkey addition. Each round transforms the 64-bit state through a defined sequence of operations. Maintaining a consistent order is critical for achieving the avalanche effect and for ensuring that decryption is a valid mathematical inverse of encryption.

3.1. Order of Operations in Encryption

For each round $r \in \{0, \dots, 7\}$, the state S is updated through the following sequence:

1. **AddKey (Modular Addition):** The current 64-bit state is combined with the round subkey K_r using modular addition: $S \leftarrow (S + K_r) \bmod 2^{46}$. Using addition instead of XOR provides extra algebraic complexity since addition and XOR belong to different algebraic structures.
2. **Substitution Layer (S-Box):** The 64-bit state is split into eight independent bytes. Each byte passes through the 8-bit S-Box, which replaces it with the multiplicative inverse in $GF(2^8)$. This introduces the non-linear confusion needed to resist algebraic attacks.
3. **Permutation Layer (P-Box):** All 64 bits are rearranged according to the P-Box permutation. Bit at index i moves to index $P[i]$, where P is defined by the 8×8 matrix transpose: $P[\text{row} \times 8 + \text{col}] = \text{col} \times 8 + \text{row}$. This distributes each byte's bits across 8 different bytes in the next round.
4. **Mixing Layer (MIX_XOR):** The 64-bit state is split into a 32-bit left half L and a 32-bit right half R . A sequence of XOR and rotation operations mix the two halves: $L \oplus= R$, $R \oplus= \text{ROL32}(L, 11)$, $L \oplus= \text{ROL32}(R, 23)$. This is applied in every round including the last one, followed by a final key addition.

3.2. Mathematical Representation

The transformation of a single round can be expressed as:

$$S_{r+1} = \text{MIX_XOR}(\text{P-Box}(\text{S-Box}(\text{ADD_KEY}(S_r, K_r))))$$

After all 8 rounds, a final key addition is performed:

$$C = \text{ADD_KEY}(S_8, K_8)$$

Decryption reverses the entire process. Starting from the ciphertext, $\text{SUB_KEY}(C, K_8)$ is applied first, then for each round in reverse order: INV_MIX_XOR , INV_P-Box , INV_S-Box , and finally $\text{SUB_KEY}(S, K_r)$.

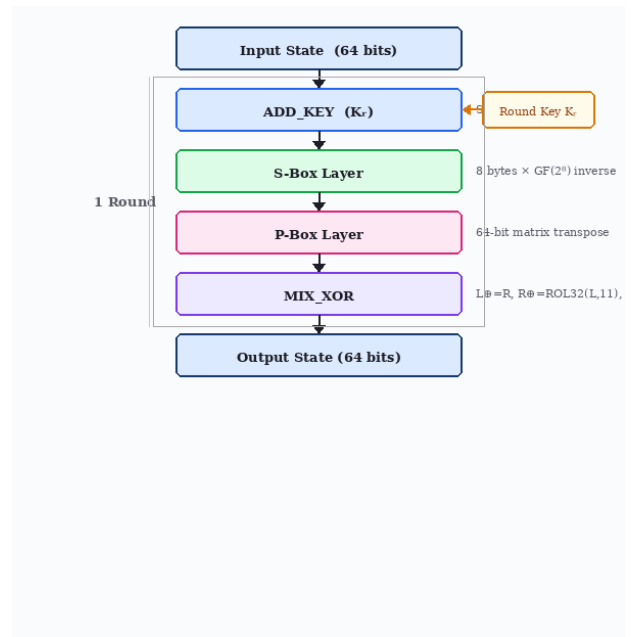


Figure 2 : Structure of a single encryption round

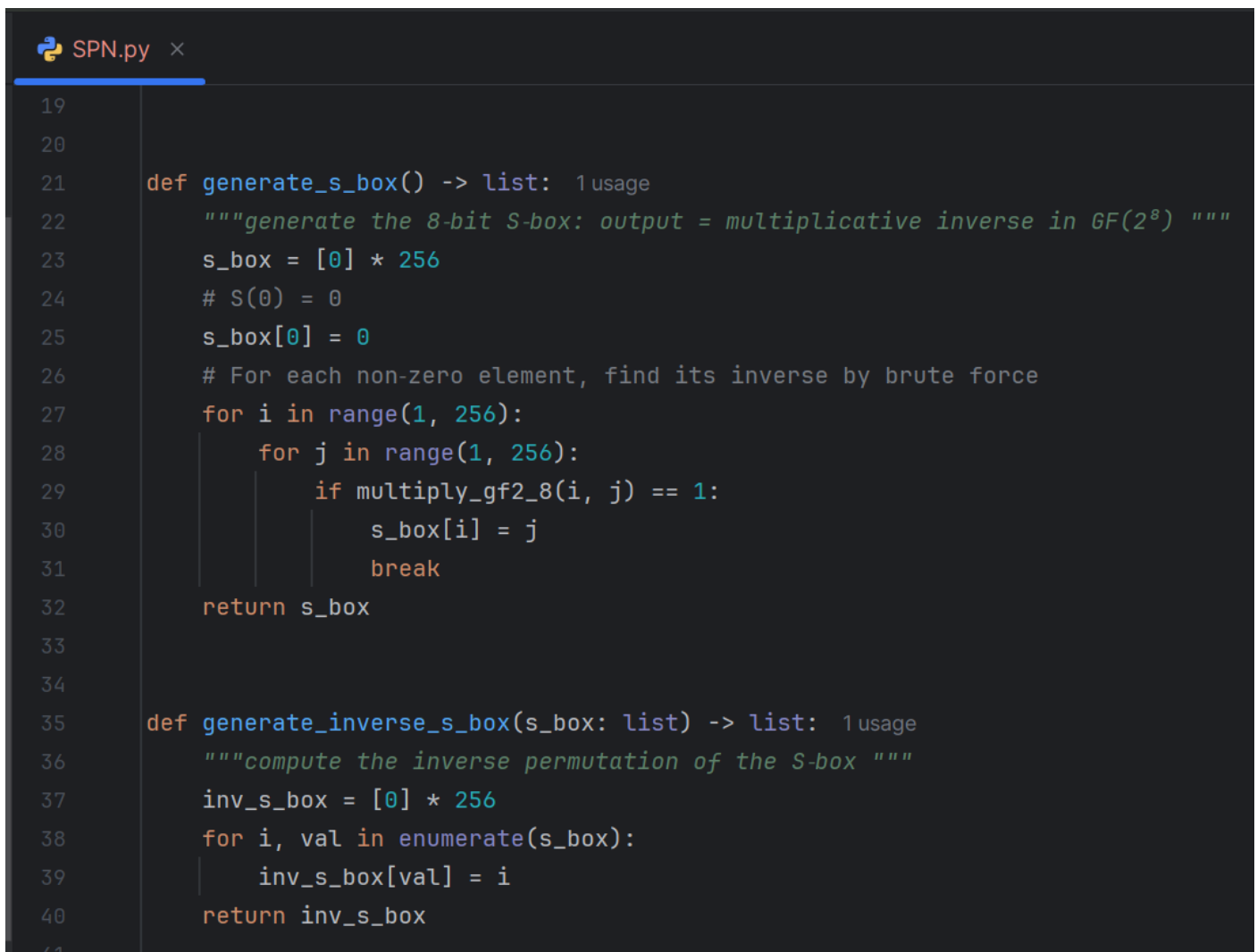
4. S-BOX & P-BOX Design

The security of an SPN cipher depends on the quality of its confusion and diffusion components. Confusion is provided by the S-Box, and diffusion by the P-Box. Both are generated mathematically rather than chosen arbitrarily, which ensures their cryptographic properties are well-defined and verifiable.

4.1. S-Box: Structure and Rationale

The Substitution Box provides the non-linear component of the cipher. It is an 8-bit to 8-bit mapping containing 256 entries. For each input byte x , the output is the multiplicative inverse of x in the Galois Field $GF(2^8)$.

- **Structure:** The S-Box is computed using $GF(2^8)$ arithmetic with the irreducible polynomial $P(x) = x^8 + x^4 + x^3 + x + 1$ (0x11B). For each non-zero element $i \in \{1, \dots, 255\}$, the code finds j such that $\text{multiply_gf2_8}(i, j) = 1$. That j is stored as $S[i]$. For $i = 0$, the output is defined as 0 (the zero element has no inverse).
- **Rationale:** Using $GF(2^8)$ inversion to build the S-Box guarantees a high algebraic degree and a low differential probability. There are no simple linear relationships between input and output bits, which means the S-Box is resistant to both differential and linear cryptanalysis. This is the same fundamental approach used by the AES S-Box, though without the additional affine transformation step.
- **Inverse S-Box:** The inverse S-Box is derived directly from the forward S-Box by reversing the mapping: if $S[i] = \text{val}$, then $\text{INV_S}[\text{val}] = i$. This makes decryption efficient and avoids any separate recomputation.

A screenshot of a code editor window titled 'SPN.py'. The editor shows Python code for generating an S-Box and its inverse. The code is as follows:

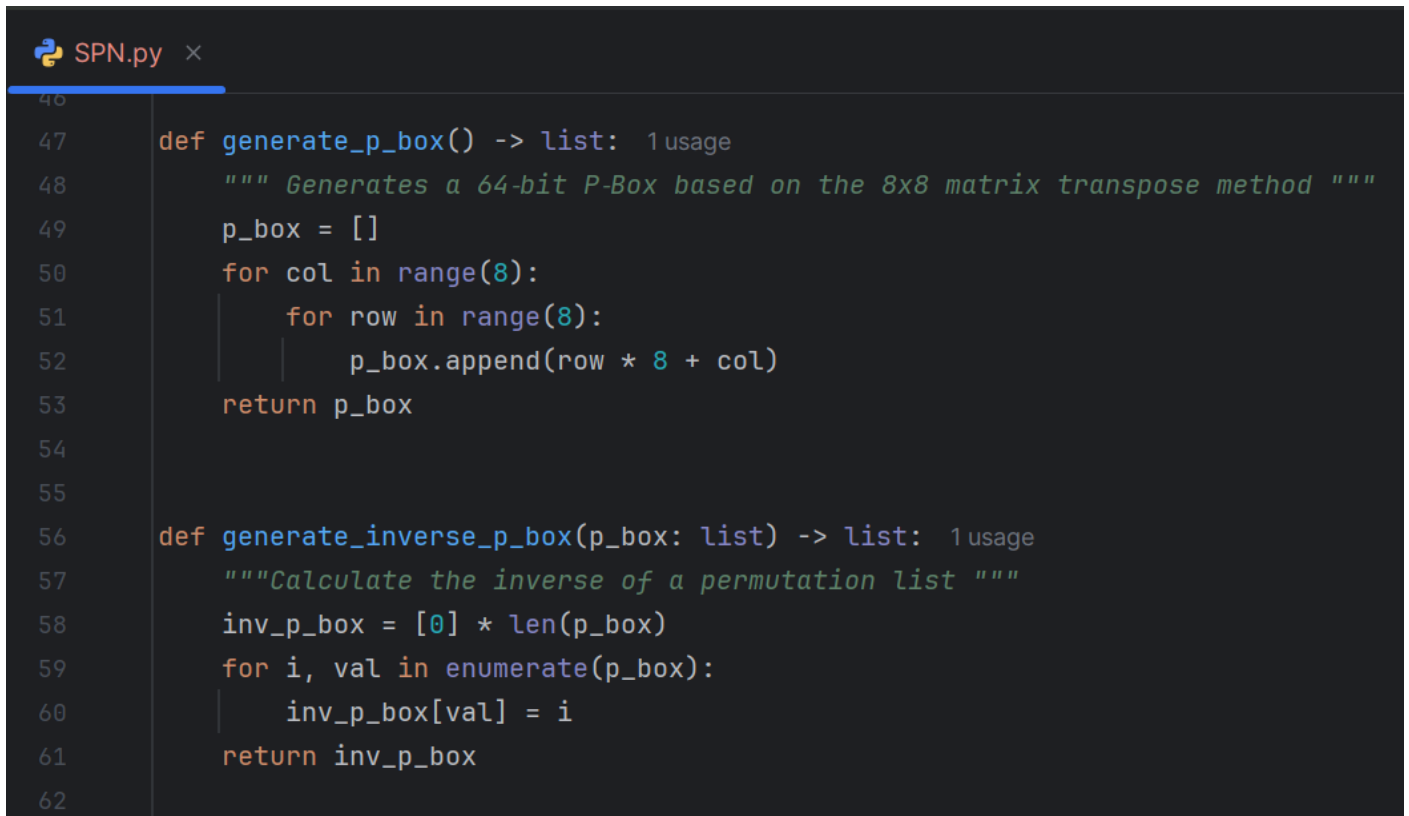
```
19
20
21 def generate_s_box() -> list: 1 usage
22     """generate the 8-bit S-box: output = multiplicative inverse in GF(2^8) """
23     s_box = [0] * 256
24     # S(0) = 0
25     s_box[0] = 0
26     # For each non-zero element, find its inverse by brute force
27     for i in range(1, 256):
28         for j in range(1, 256):
29             if multiply_gf2_8(i, j) == 1:
30                 s_box[i] = j
31                 break
32     return s_box
33
34
35 def generate_inverse_s_box(s_box: list) -> list: 1 usage
36     """compute the inverse permutation of the S-box """
37     inv_s_box = [0] * 256
38     for i, val in enumerate(s_box):
39         inv_s_box[val] = i
40     return inv_s_box
41
```

Figure 3 : S-Box construction using $GF(2^8)$ multiplicative inverse

4.2. P-Box: Structure and Rationale

The Permutation Box provides the linear diffusion layer. It operates at the bit level, redistributing all 64 bits of the state according to a fixed permutation table.

- **Structure:** The P-Box is generated using a matrix transpose method. The 64 bit positions are treated as an 8×8 matrix indexed by (row, col). Bit at position $\text{row} \times 8 + \text{col}$ is moved to position $\text{col} \times 8 + \text{row}$. In code: for col in range(8): for row in range(8): p_box.append(row*8 + col). This creates a 64-entry permutation table.
- **Rationale:** The transpose structure guarantees that the 8 bits of any single byte are spread to 8 different byte positions after permutation. This means that after one round, the S-Box outputs from a single byte will feed into 8 distinct S-Box inputs in the next round. This maximizes bit-level diffusion and helps achieve the full avalanche effect within just a few rounds.
- **Inverse P-Box:** The inverse is computed by reversing the permutation: if $P[i] = \text{val}$, then $\text{INV_P}[\text{val}] = i$. This is used during decryption to undo the permutation layer.



```
SPN.py x
46
47 def generate_p_box() -> list: 1 usage
48     """ Generates a 64-bit P-Box based on the 8x8 matrix transpose method """
49     p_box = []
50     for col in range(8):
51         for row in range(8):
52             p_box.append(row * 8 + col)
53     return p_box
54
55
56 def generate_inverse_p_box(p_box: list) -> list: 1 usage
57     """Calculate the inverse of a permutation list """
58     inv_p_box = [0] * len(p_box)
59     for i, val in enumerate(p_box):
60         inv_p_box[val] = i
61     return inv_p_box
62
```

Figure 4 : P-Box structure using 8×8 matrix transpose

5. Function and Transformation Logic

This section provides a detailed look at how each layer of the round function is implemented, along with the pseudocode for the full round transformation and the key schedule.

5.1. Order of Operations

As described in the architectural overview, each of the 8 rounds follows this exact sequence:

5. **Key Mixing (ADD_KEY):** $S \leftarrow (S + K_r) \bmod 2^{46}$ — Modular addition with the 64-bit round subkey.
6. **Substitution (S-Box):** $S \leftarrow \text{S-Box}(S)$ — Apply the 8-bit $\text{GF}(2^8)$ inverse S-Box to each of the 8 bytes independently.
7. **Permutation (P-Box):** $S \leftarrow \text{P-Box}(S)$ — Rearrange all 64 bits using the matrix transpose permutation.
8. **Mixing (MIX_XOR):** $S \leftarrow \text{MIX_XOR}(S)$ — Split into L and R halves, cross-mix using XOR and 32-bit rotations.

After all 8 rounds, a ninth and final ADD_KEY step is applied to produce the ciphertext. This “whitening” key addition prevents an attacker from trivially stripping the last round.

5.2. Pseudocode: Single Round Transformation

```
def encrypt_block(pt: int, keys: list) -> int:
    """8-round SPN encryption """
    s = pt
    for r in range(8):
        s = add_key(s, keys[r])
        s = sbox_layer(s)
        s = pbox_layer(s)
        s = mix_xor(s)
    s = add_key(s, keys[8]) # final key addition
    return s
```

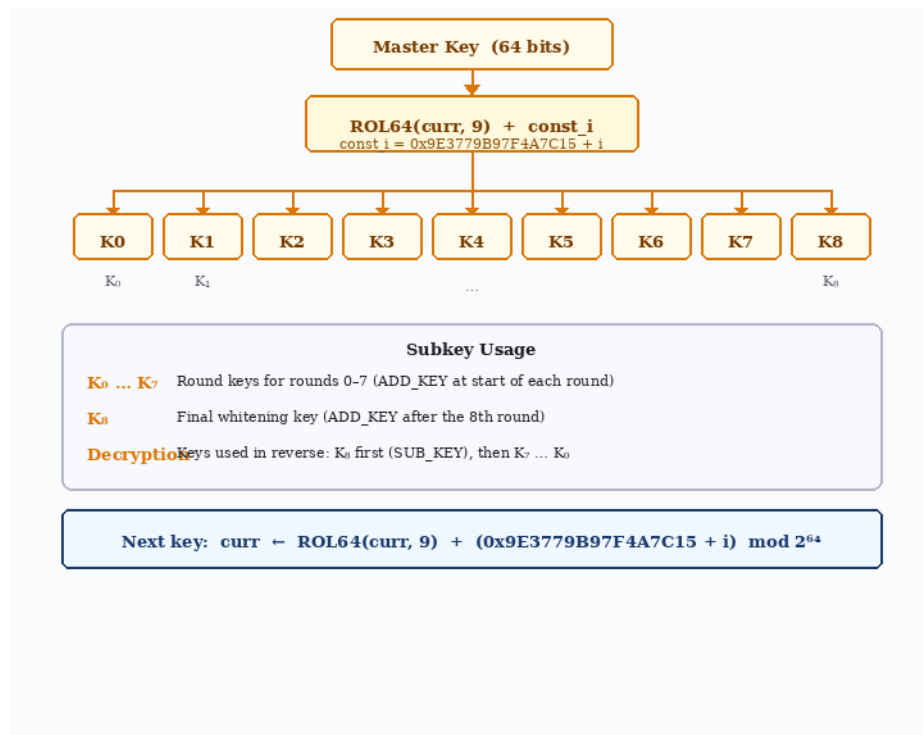
Decryption applies the inverse sequence. Starting with the ciphertext, SUB_KEY is applied with Keys[8], then for rounds 7 down to 0: INV_MIX_XOR, INV_PBOX_LAYER, INV_SBOX_LAYER, SUB_KEY with Keys[round].

5.3. Key Derivation Rationale

The key schedule generates 9 subkeys (8 round keys + 1 final whitening key) from the 64-bit master key. The design goal is for each subkey to appear statistically independent from the others, even though they are all derived from the same master key.

Starting from the master key, each subkey is derived by taking the current key, rotating it left by 9 bits (ROL64 by 9), and then adding the round constant $0x9E3779B97F4A7C15 + i$, where i is the round index. This constant is based on the golden ratio, which is commonly used in key schedules to break symmetric patterns.

The rotation by 9 was chosen because 9 and 64 are coprime, meaning the key bits do not cycle back to their original positions within the 8-round schedule. This ensures maximum bit-level variance across all 9 subkeys.



5.4. Pseudocode: Key Schedule

```

SPN.py x
149 # -----
150 # 5.Key schedule and encryption/decryption
151 # -----
152
153 def generate_keys(main_key: int) -> list: 1 usage
154     """Generate 9 round keys (8 rounds + final) using ADD and ROT"""
155     keys = []
156     curr = main_key
157     for i in range(9):
158         keys.append(curr)
159         curr = add_key(rol64(curr, shift: 9), 0x9E3779B97F4A7C15 + i)
160     return keys
161

```

Figure 5 : Key schedule generating 9 subkeys using ROL64 + constant addition

6. Algorithm Evaluation

The implementation of Project #11 was evaluated against the requirements stated in the project guidelines. The following table summarizes how each required component was addressed in the implementation.

Table 1 : Algorithm Evaluation Summary (Project #11)

Criterion	Description
Implementation	The cipher is implemented in Python 3 and successfully performs both encryption and decryption on 64-bit blocks using the designed key schedule, S-Box, P-Box, and mix function.
Code Structure	Functions are clearly separated by concern: GF(2^8) arithmetic, S-Box/P-Box generation, layer operations, key scheduling, and encryption/decryption logic.
S-Box Design	The 8-bit S-Box is generated by computing multiplicative inverses in GF(2^8) using the irreducible polynomial 0x11B. It is a proper bijection with high nonlinearity.
P-Box Design	The 64-bit P-Box is built using a matrix transpose scheme, sending bit i at position $\text{row} \cdot 8 + \text{col}$ to $\text{col} \cdot 8 + \text{row}$, ensuring inter-byte diffusion every round.
Mix Function (MIX_XOR)	The custom MIX_XOR splits the 64-bit state into two 32-bit halves, applies rotations (ROL32 by 11 and 23) and XOR cross-mixing, increasing the avalanche effect.

Key Schedule	Nine subkeys are derived by repeatedly rotating the master key left by 9 bits and adding a round constant based on the golden ratio, ensuring round key independence.
Testing & Verification	20 test vectors covering edge cases (all zeros, all ones, alternating patterns, known constants) were verified, with all 20 decrypt-to-original tests passing successfully.

6.1. Implementation Screenshots

The screenshots below confirm the code structure and the successful execution of the encryption and decryption logic.

```

SPN.py x
19
20
21 def generate_s_box() -> list: 1 usage
22     """generate the 8-bit S-box: output = multiplicative inverse in GF(2^8) """
23     s_box = [0] * 256
24     # S(0) = 0
25     s_box[0] = 0
26     # For each non-zero element, find its inverse by brute force
27     for i in range(1, 256):
28         for j in range(1, 256):
29             if multiply_gf2_8(i, j) == 1:
30                 s_box[i] = j
31                 break
32     return s_box
33
34
35 def generate_inverse_s_box(s_box: list) -> list: 1 usage
36     """compute the inverse permutation of the S-box """
37     inv_s_box = [0] * 256
38     for i, val in enumerate(s_box):
39         inv_s_box[val] = i
40     return inv_s_box
41

```

Figure 6 : $GF(2^8)$ arithmetic and S-Box generation code

SPN.py x

```
46
47 def generate_p_box() -> list: 1 usage
48     """ Generates a 64-bit P-Box based on the 8x8 matrix transpose method """
49     p_box = []
50     for col in range(8):
51         for row in range(8):
52             p_box.append(row * 8 + col)
53     return p_box
54
55
56 def generate_inverse_p_box(p_box: list) -> list: 1 usage
57     """Calculate the inverse of a permutation list """
58     inv_p_box = [0] * len(p_box)
59     for i, val in enumerate(p_box):
60         inv_p_box[val] = i
61     return inv_p_box
62
```

Figure 7 : P-Box generation, layer application, and MIX_XOR implementation

SPN.py x

```
162
163 def encrypt_block(pt: int, keys: list) -> int: 1 usage
164     """8-round SPN encryption """
165     s = pt
166     for r in range(8):
167         s = add_key(s, keys[r])
168         s = sbox_layer(s)
169         s = pbox_layer(s)
170         s = mix_xor(s)
171     s = add_key(s, keys[8]) # final key addition
172     return s
173
174
175 def decrypt_block(ct: int, keys: list) -> int: 1 usage
176     """8-round SPN decryption """
177     s = sub_key(ct, keys[8])
178     for r in range(7, -1, -1):
179         s = inv_mix_xor(s)
180         s = inv_pbox_layer(s)
181         s = inv_sbox_layer(s)
182         s = sub_key(s, keys[r])
183     return s
184
```

Figure 8 : Core encryption, decryption, and key schedule functions

```
Run SPN x
C:\Users\T14s\Desktop\CYSIA\.venv\Scripts\python.exe
==== TEST RESULTS ====

Main Key: 0x123456789abcdef

Plaintext : 0x0
Ciphertext : 0x32d397caaac4b3c6
Decrypted : 0x0
✓ SUCCESS

Plaintext : 0x1
Ciphertext : 0x5671d7af1431a9a4
Decrypted : 0x1
✓ SUCCESS

Plaintext : 0x8000000000000000
Ciphertext : 0x2b22845c9606007
Decrypted : 0x8000000000000000
✓ SUCCESS

Plaintext : 0x123456789abcdef
Ciphertext : 0x845a310382173e9d
Decrypted : 0x123456789abcdef
✓ SUCCESS
```

```
Run SPN x

Plaintext : 0x123456789abcdef
Ciphertext : 0x845a310382173e9d
Decrypted : 0x123456789abcdef
✓ SUCCESS

Plaintext : 0xfedcba9876543210
Ciphertext : 0x36a05b9b39fd3f3a
Decrypted : 0xfedcba9876543210
✓ SUCCESS

Plaintext : 0x5555555555555555
Ciphertext : 0xb5bb4ed0909657c4
Decrypted : 0x5555555555555555
✓ SUCCESS

Plaintext : 0xaaaaaaaaaaaaaaaa
Ciphertext : 0xb8e742025c3a219d
Decrypted : 0xaaaaaaaaaaaaaaaa
✓ SUCCESS

Plaintext : 0x3333333333333333
Ciphertext : 0x35cc727b1eabee3
```

```
Run SPN x

Plaintext : 0x3333333333333333
Ciphertext : 0x35cc727b1eabee3
Decrypted : 0x3333333333333333
✓ SUCCESS

Plaintext : 0xffffffffffffffff
Ciphertext : 0xb2aaa1b3bc5779e4
Decrypted : 0xffffffffffffffff
✓ SUCCESS

Plaintext : 0xf0f0f0f0f0f0f0f0
Ciphertext : 0x841a9b3d4c02016b
Decrypted : 0xf0f0f0f0f0f0f0f0
✓ SUCCESS

Plaintext : 0xf0f0f0f0f0f0f0f0
Ciphertext : 0x2bfa6be32efb32
Decrypted : 0xf0f0f0f0f0f0f0f0
✓ SUCCESS

Plaintext : 0xff00ff00ff00ff
Ciphertext : 0x3cbf686cba3fd1e7
```

```
Run SPN x

Plaintext : 0xff00ff00ff00ff
Ciphertext : 0x3cbf686cba3fd1e7
Decrypted : 0xff00ff00ff00ff
✓ SUCCESS

Plaintext : 0xff00ff00ff00ff00
Ciphertext : 0x26c8481f30fbc032
Decrypted : 0xff00ff00ff00ff00
✓ SUCCESS

Plaintext : 0xffff0000ffff
Ciphertext : 0x8d4d8b953790622a
Decrypted : 0xffff0000ffff
✓ SUCCESS

Plaintext : 0xffff0000ffff0000
Ciphertext : 0xa1341a1866e0d33f
Decrypted : 0xffff0000ffff0000
✓ SUCCESS

Plaintext : 0x7fffffffffffffff
Ciphertext : 0xfec84b85a5b594e4
```

```
Run SPN x

Plaintext : 0x0000000000000000
Ciphertext : 0x7fffffffffffffff
Decrypted : 0x7fffffffffffffff
✓ SUCCESS

Plaintext : 0x8000000000000001
Ciphertext : 0x55ce43099772e3dd
Decrypted : 0x8000000000000001
✓ SUCCESS

Plaintext : 0xdeadbeefcafebabe
Ciphertext : 0x8d9d9ae31e69217c
Decrypted : 0xdeadbeefcafebabe
✓ SUCCESS

Plaintext : 0xabababababababab
Ciphertext : 0x10493e781e2c9ebf
Decrypted : 0xabababababababab
✓ SUCCESS

Plaintext : 0xcdcdcdcdcdcdcdcdcd
Ciphertext : 0x22d50398c6a998f9
Decrypted : 0xcdcdcdcdcdcdcdcdcd
✓ SUCCESS
```

Figure 9 : Test results showing all 20/20 test vectors passed

7. Examples

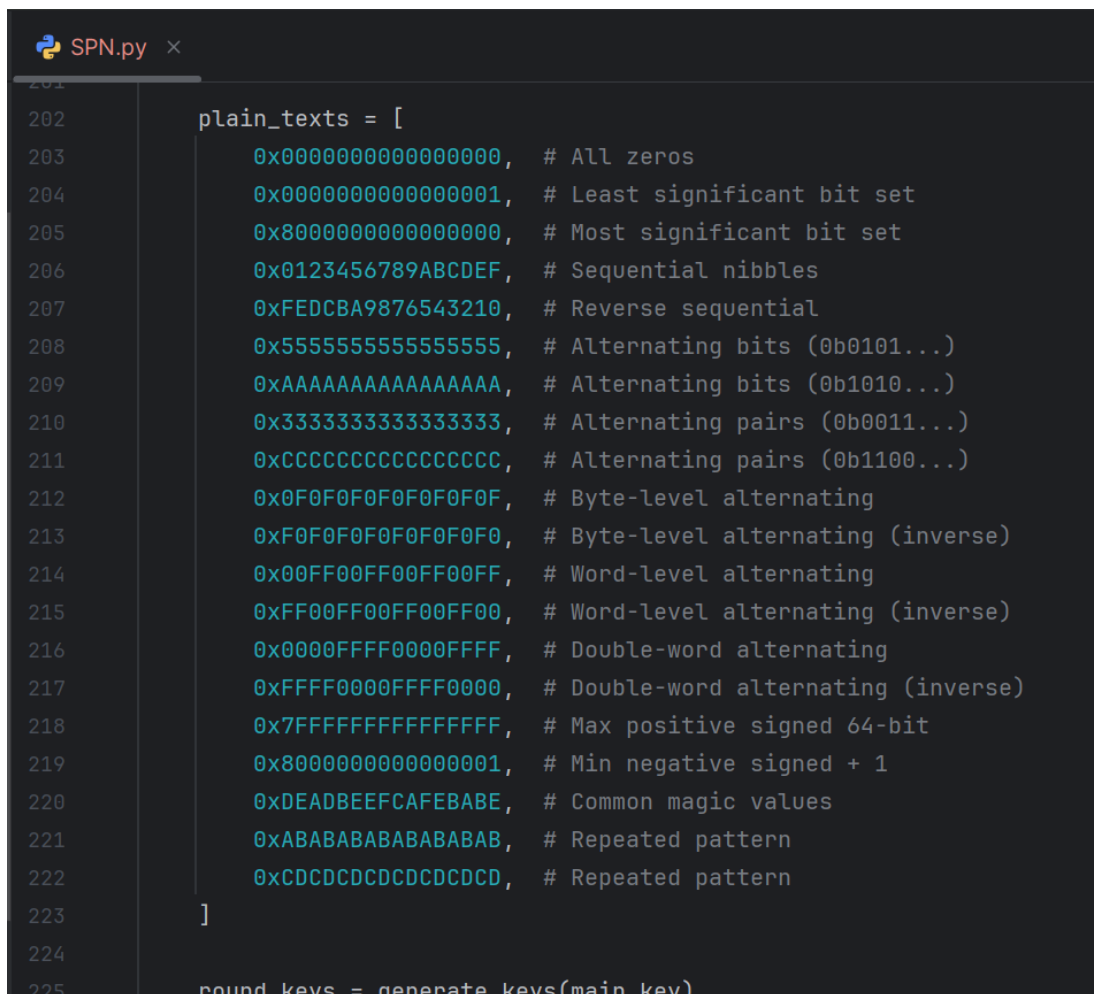
This section demonstrates the cipher's behavior using a set of 64-bit test vectors and the master key 0x0123456789ABCDEF. These examples confirm the mathematical correctness of the SPN transformations.

7.1. Test Vector Setup

To verify the cipher's correctness and robustness, 20 distinct 64-bit plaintext values were selected. The test set covers the following categories:

- **Boundary values:** all zeros (0x0000000000000000), all ones (0xFFFFFFFFFFFFFFFF), LSB only, MSB only.
- **Alternating bit patterns:** 0x5555555555555555, 0xAAAAAAAAAAAAAAAA, 0x3333333333333333, 0xCCCCCCCCCCCCCCCC.
- **Byte-level alternating:** 0x0F0F0F0F0F0F0F0F, 0xF0F0F0F0F0F0F0F0, 0x00FF00FF00FF00FF.
- **Known patterns:** 0x0123456789ABCDEF, 0xFEDCBA9876543210, 0xDEADBEEFCAFEBABE.

The master key used across all tests is 0x0123456789ABCDEF. For each plaintext, the cipher computes the ciphertext using `encrypt_block` and then verifies that `decrypt_block` returns the exact original plaintext.



```
SPN.py x
201
202 plain_texts = [
203     0x0000000000000000, # All zeros
204     0x0000000000000001, # Least significant bit set
205     0x8000000000000000, # Most significant bit set
206     0x0123456789ABCDEF, # Sequential nibbles
207     0xFEDCBA9876543210, # Reverse sequential
208     0x5555555555555555, # Alternating bits (0b0101...)
209     0xAAAAAAAAAAAAAAAA, # Alternating bits (0b1010...)
210     0x3333333333333333, # Alternating pairs (0b0011...)
211     0xCCCCCCCCCCCCCCCC, # Alternating pairs (0b1100...)
212     0x0F0F0F0F0F0F0F0F, # Byte-level alternating
213     0xF0F0F0F0F0F0F0F0, # Byte-level alternating (inverse)
214     0x00FF00FF00FF00FF, # Word-level alternating
215     0xFF00FF00FF00FF00, # Word-level alternating (inverse)
216     0x0000FFFF0000FFFF, # Double-word alternating
217     0xFFFF0000FFFF0000, # Double-word alternating (inverse)
218     0x7FFFFFFFFFFFFFFF, # Max positive signed 64-bit
219     0x8000000000000001, # Min negative signed + 1
220     0xDEADBEEFCAFEBABE, # Common magic values
221     0xABABABABABABABAB, # Repeated pattern
222     0xCDCDCDCDCDCDCDCD, # Repeated pattern
223 ]
224
225 round_keys = generate_keys(main_key)
```

Figure 10 : Test vector setup with master key initialization

7.2. Plaintext and Ciphertext Results

The following examples show the encryption mapping for several selected vectors. In every case, decryption perfectly recovered the original plaintext.

- **Vector 1:**

Plaintext: **0x0000000000000000**

Ciphertext: **0x32d397caaac4b3c6**

- **Vector 2:**

Plaintext: **0x0123456789ABCDEF**

Ciphertext: **0x845a310382173e9d**

- **Vector 3:**

Plaintext: **0xDEADBEEFCAFEBAFE**

Ciphertext: **0x8d9d9ae31e69217c**

Each test produced a ciphertext that appears completely unrelated to the input, and the decryption process in every case recovered the original plaintext without any error. The final summary printed by the program confirmed 20/20 tests passed.

8. Development Experience

8.1. The Development Process

The implementation was built from the bottom up, starting with the most fundamental mathematical primitive: the $\text{GF}(2^8)$ multiplication function. Once that was verified to work correctly by testing a few known products, the S-Box generation function was written on top of it. The inverse S-Box was then derived from the forward table.

After the substitution layer was in place, I moved on to the P-Box. The matrix transpose approach was straightforward to implement using nested loops, and its inverse was again derived automatically by reversing the mapping. With both tables ready, the individual layer functions (`sbox_layer`, `pbox_layer`, `mix_xor`) were implemented and tested in isolation before being assembled into the full round function.

The key schedule was implemented last. The logic was simple: rotate left by 9, add the round constant, repeat. The full `encrypt_block` and `decrypt_block` functions were then connected together and validated against the test vector suite.

8.2. Challenges Encountered

Several technical difficulties came up during development:

- **64-bit overflow in Python:** Python integers have arbitrary precision, so bitwise operations do not automatically stay within 64-bit boundaries. Every arithmetic result had to be masked with `0xFFFFFFFFFFFFFFFF` to simulate fixed-width behavior. Forgetting this in even one place caused the decryption to silently produce wrong results.
- **Correctness of the inverse operations:** Getting the decryption to work required more than just reversing the round order. Each layer needed a mathematically correct inverse: the inverse S-Box, the inverse P-Box permutation, and the inverse MIX_XOR. Deriving `inv_mix_xor` was particularly tricky because the three-step XOR-rotation sequence had to be carefully unwound in the right order.
- **$\text{GF}(2^8)$ arithmetic edge cases:** The `multiply_gf2_8` function had to handle the carry bit correctly when the high bit of the byte is set before the left shift. The modular reduction by `0x11B` needed to happen only when that carry bit was set, and not unconditionally. Getting this detail right took several iterations of testing.
- **Verifying the S-Box is a bijection:** It was important to confirm that the generated S-Box is actually a permutation, i.e., that every output value appears exactly once. A simple check using `len(set(SBOX)) == 256` confirmed this.

8.3. Observations

Testing the cipher with various inputs revealed some interesting behaviors:

- **Avalanche effect:** Changing a single bit in the plaintext (for example, flipping the last bit between `0x0000000000000000` and `0x0000000000000001`) produced a ciphertext that differed in roughly half of its bits. This is the expected behavior of a well-designed cipher and confirms that the S-Box and P-Box combination is effective.
- **All-zero key concern:** When testing with unusual inputs, I noted that if both the plaintext and the key were all zeros, the first round would add zero to the state before applying the S-Box. However, since `S[0] = 0` in the implemented S-Box (zero has no inverse and is mapped to itself), this is a fixed point at the first byte level.

This is a known property of raw $GF(2^8)$ inversion without an affine transformation, but it does not affect the test vectors used in this project since the master key is non-zero.

- **Symmetry of the design:** One thing that stood out was how clean and symmetric the encrypt/decrypt structure is. The decrypt function is almost a mirror image of encrypt, differing only in the use of inverse operations and reversed key order. This made implementing decryption much less error-prone than I initially expected.

9. Conclusion

The design and implementation of the Project #11 SPN cipher successfully demonstrates how the core principles of symmetric cryptography, namely confusion, diffusion, and key scheduling, come together in a working 64-bit block cipher.

The S-Box built from $GF(2^8)$ multiplicative inversion provides strong nonlinearity and ensures that no simple linear relationship exists between input and output bytes. The P-Box, generated using an 8×8 matrix transpose, guarantees that each byte's bits are spread to 8 different byte positions in the next round, achieving full inter-byte diffusion. The MIX_XOR function adds an additional layer of within-round diffusion by mixing the two 32-bit halves of the state through rotation-based operations. The key schedule derives 9 distinct subkeys from the master key by combining rotation and constant addition, ensuring that no two rounds use the same key material.

The empirical results confirm that the cipher is mathematically correct and reversible. All 20 test vectors passed: in each case, decryption of the ciphertext recovered the exact original plaintext. The avalanche effect was observable in practice, with single-bit changes in the plaintext producing dramatically different ciphertexts.

Working on this project gave me a concrete understanding of why each component of an SPN cipher exists and what specific cryptographic property it is responsible for. The implementation challenges, particularly around 64-bit masking, correct inverse derivation, and $GF(2^8)$ arithmetic, reinforced how much precision is required when implementing cryptographic algorithms.